

Inference-Sufficient Representations for High-Throughput Measurement: Lessons from Lossless Compression Benchmarks in 4D-STEM

Ondrej Dyck^{1,*}
Andrew R. Lupini¹
Albina Borisevich¹
Miaofang Chi¹
Rama K. Vasudevan¹
Stephen Jesse¹

¹Center for Nanophase Materials Sciences, Oak Ridge National Laboratory, Oak Ridge, TN 37831, USA

*Corresponding author: dyckoe@ornl.gov

Abstract

Four-dimensional scanning transmission electron microscopy (4D-STEM) generates datasets whose volume already outpaces practical storage, transfer and interactive analysis. We benchmark 13 off-the-shelf lossless compression implementations across five representative datasets and find that the Blosc family substantially outperforms the compressors built into HDF5, achieving comparable compression ratios at far higher throughput; this comparison is intended as practical guidance for choosing an implementation. The broader conclusion, however, is that this route is closing. Lossless compression is bounded by the entropy of the source, and on a simulated dataset where that bound can be computed exactly the best implementation already comes within a factor of 2.5 of the entropy bound. Whatever a better implementation recovers will be a small multiple, not the orders of magnitude that rising detector rates demand. What must be decided instead is which data to keep and which to discard. This is a judgement researchers already make routinely and often intuitively, and one that will have to be made explicit as autonomous and agentic systems take on data collection, analysis and reduction. We argue for inference-sufficient representations: a record suffices when it retains what is required to support an intended inference, not when it is complete. We distinguish coarsening, which cannot be false, from interpretation, which can, and show that event-based detectors already perform the latter. Design principles are suggested for deciding what data representation must be preserved to support a given inference.

1 Introduction

Four-dimensional scanning transmission electron microscopy (4D-STEM) has emerged as a powerful family of techniques for characterizing materials by recording a diffraction pattern at each probe position in a 2D scan, enabling downstream reconstruction of virtual imaging modes and quantitative analysis beyond conventional STEM signals [1]. The resulting datasets are information-rich but also large, and they increasingly expose a mismatch between acquisition bandwidth and practical resources for storage, transfer, and interactive analysis. Importantly, this mismatch is not unique to any one modality: as detector performance improves across electron and photon sciences, the rate at which experiments can generate data is growing faster than the infrastructure typically available to move and retain it. Recent detector and acquisition system designs explicitly target very high sustained data rates and motivate specialized pipelines to ingest and process streams ranging from tens of Gb s^{-1} to tens of GB s^{-1} [2, 3, 4, 5, 6].

In this work we use 4D diffraction and momentum-resolved electron energy loss spectroscopy (EELS) datasets as concrete, representative examples of this broader data-management problem. In what follows, we use “4D-EELS” as a shorthand for momentum-resolved EELS acquired over a 2D scan, producing datasets

with two spatial dimensions plus energy and momentum (scattering-angle) dimensions. Such datasets can exceed the size of comparably sampled 4D diffraction datasets because the energy-dispersion axis demands a larger detector extent along the spectral dimension. For example, while a typical 4D diffraction dataset may record 256×256 detector pixels per scan position, our Merlin detector in EELS mode records 256×1024 pixels per scan position, increasing the per-position data volume by a factor of four. These examples illustrate a general point: as experimentalists push toward higher throughput, higher dynamic range, and higher resolution, data volume becomes a first-order experimental constraint rather than a downstream nuisance.

Lossless compression provides an immediate and widely applicable baseline for reducing this burden. In the HDF5 ecosystem, the gzip family is broadly available and often serves as a default; gzip-9 typically provides the strongest compression among the built-in options but can be prohibitively slow for high-throughput workflows. This motivates a practical question: are there drop-in compression implementations that provide comparable compression ratios while substantially improving write and read performance, without changing file formats or compromising numerical fidelity?

Here we present a systematic benchmark of 13 readily available, lossless compression implementations for representative 4D-STEM datasets. We quantify compression ratio, write throughput, read throughput, and reproducibility across 10 independent runs, spanning five datasets (8 MiB to 8 GiB) and multiple chunking strategies. Our goal is pragmatic: to provide implementation-level guidance for selecting compression filters that improve storage efficiency and I/O performance in common Python/HDF5 workflows.

Beyond implementation-level performance, this benchmark highlights a broader implication for high-throughput microscopy: lossless compression is valuable, but it is not a complete solution. As detector throughput continues to rise, lossless compression can reduce—but cannot eliminate—the mismatch between acquisition rates and practical storage and I/O budgets. In throughput-limited regimes, sustainable workflows will increasingly depend on inference-driven data reduction and interpretation pipelines that store task-relevant reduced representations rather than fully dense raw measurements. Event-based representations provide one concrete example of this principle in high-rate microscopy workflows [7].

2 Methods

2.1 Datasets

We evaluated compression performance on five representative datasets spanning different acquisition modes, detector configurations, and sparsity levels (Table 1).

Table 1: Datasets used for compression benchmarking

Dataset	Shape	Size	Sparsity	Description
4D_EELS	(64, 64, 256, 1024)	2.0 GiB	92.8%	Full 4D EELS spectrum imaging
4D_Diff	(256, 256, 256, 256)	8.0 GiB	74.7%	4D STEM diffraction (unbinned)
4D_Diff-2x2	(256, 256, 128, 128)	4.0 GiB	70.9%	4D STEM diffraction (2×2 binned)
4D_Diff-4x4	(256, 256, 64, 64)	1.0 GiB	60.9%	4D STEM diffraction (4×4 binned)
3D_EELS	(64, 64, 1, 1024)	8.0 MiB	49.5%	Y-summed EELS spectrum image

All datasets were acquired using a Quantum Detectors MerlinEELS detector [8] and initially stored in the native `.mib` format [9] before conversion to the HDF5-based EMD 1.0 (Electron Microscopy Dataset) format [10]. The datasets span a $\sim 10^3$ -fold range in size (8.0 MiB to 8.0 GiB) and represent typical experimental conditions. The high sparsity (49.5–92.8% zeros) is characteristic of electron diffraction and spectroscopy data, where most detector pixels record no signal. The raw 4D datasets (4D_Diff and 4D_EELS) are stored as `uint16` and, for these acquisitions, exhibit an effective dynamic range of 12 bits (maximum observed value 4095). The binned diffraction datasets were generated by averaging and stored as `float32`. The 3D_EELS dataset is a y-summed (integrated) spectrum image and therefore makes fuller use of the `uint16` container range. The binned datasets enable evaluation of how detector binning affects both file size and compression performance.

2.2 Compression Implementations

We evaluated 13 off-the-shelf lossless compression implementations spanning three categories: HDF5 built-in methods, advanced compression via the hdf5plugin library, and sparse matrix storage, together with two custom strategies of our own that ultimately underperformed (Table 2). All off-the-shelf methods are available in standard Python scientific computing libraries, ensuring reproducibility and ease of adoption.

Several of these implementations share an underlying compression algorithm. DEFLATE [11] is used by gzip at all three effort levels, by Blosc Zlib, and by our direct compression of a numpy array; the LZ4 format is produced by standalone LZ4, by Blosc LZ4 and LZ4HC, and by Bitshuffle+LZ4. What distinguishes these implementations is therefore not the compression algorithm but the preprocessing applied beforehand (byte- or bit-shuffling), the blocking and threading strategy, and the degree of integration with HDF5. Because a practitioner selects an implementation rather than an algorithm, this study is framed as a comparison of implementations; the algorithmic details of each codec are documented in the references cited in Table 2.

Table 2: Compression implementations evaluated in this study

Implementation	Variant	Characteristics & Use Case	Ref.
<i>HDF5 Built-in Compression</i>			
gzip	level 1	Fast DEFLATE; minimal overhead	[11]
	level 6	Balanced; general-purpose default	[11]
	level 9	Maximum compression; archival	[11]
LZF	-	Fast HDF5-optimized; interactive use	[12]
szip	-	Extended-Rice algorithm; scientific data	[13]
<i>Advanced Compression (Blosc)</i>			
Blosc	blosclz	Native codec; byte-shuffling	[14]
	lz4	Ultra-fast; real-time applications	[15]
	lz4hc	High-compression LZ4 variant	[15]
	zlib	Standard zlib in Blosc framework	[11]
	zstd	Modern; excellent balance	[16]
<i>Other Advanced Methods</i>			
LZ4	standalone	Extremely fast decompression	[15]
Bitshuffle+LZ4	-	Bit-shuffling; sparse data	[17]
<i>Sparse Matrix Storage</i>			
CSR	-	Compressed Sparse Row; >95% sparse	[18]
<i>Custom Strategies</i>			
uint8+overflow	-	8-bit downcast; low counts	-
Simple gzip	-	Direct array compression	[11]

HDF5 built-in methods (gzip, LZF, szip) provide baseline performance and are universally supported in the HDF5 ecosystem [19]. Advanced methods via hdf5plugin include the Blosc meta-compressor framework with five codec variants (blosclz, lz4, lz4hc, zlib, zstd), standalone LZ4 optimized for extremely fast decompression, and Bitshuffle which performs bit-level shuffling before compression to improve performance on numerical data. All Blosc-provided implementations use byte-shuffling and blocking to reorder data for better compression of numerical arrays.

Sparse matrix storage uses SciPy’s Compressed Sparse Row (CSR) format, storing each diffraction pattern as a sparse matrix. This approach is particularly suitable for highly sparse datasets (>95% zeros) but requires custom loading code and does not integrate with standard HDF5 workflows.

When considering data reduction strategies a natural question that arises is whether efficiency may be found by simply reducing the bit depth of the data. A custom strategy we attempted includes uint8 downcasting with overflow handling, which exploits the fact that most detector pixels have low counts (<255) and can be stored in 8-bit format with a separate overflow map for rare high-count pixels. We find that this strategy is significantly worse than standard compression implementations. Simple gzip compression of numpy arrays provides a baseline comparison without HDF5 overhead.

2.3 Chunking Strategies

HDF5 chunking divides the 4D dataset into smaller blocks that are compressed independently. Chunk size affects both compression ratio and access performance. We tested three chunking strategies optimized for different access patterns. Real-space optimized chunks (32, 32, at most 256×256) favor accessing contiguous scan regions for virtual imaging and spatial feature analysis. Balanced chunks (16, 16, at most 128×128) provide a compromise between scan-space and detector-space access. Single-frame chunks (1, 1, full detector) optimize for accessing individual diffraction patterns. The detector dimensions in each chunk are capped at the actual detector configuration (e.g., 256×256 for unbinned data, 128×128 for 2×2 binned data), ensuring valid HDF5 configurations across all datasets. Each HDF5 compression implementation was tested with all three chunking strategies, yielding 39 HDF5 method combinations plus sparse and custom methods.

2.4 Benchmark Methodology

2.4.1 Performance Metrics

For each implementation and dataset combination, we measured compression ratio (original size divided by compressed size), write time (time to compress and save the entire 4D dataset), read time (time to load and decompress the entire 4D dataset), and file size (actual disk space used including HDF5 metadata overhead). Write throughput was calculated from the original uncompressed data size divided by write time. Read throughput was calculated from the decompressed data size (bytes read into memory) divided by read time. Compression ratio directly impacts storage costs and network transfer times. Write time represents the one-time preprocessing cost; while write times of several minutes are acceptable for archival storage, faster compression is preferred for interactive workflows. Read time is critical for data analysis workflows where the entire dataset must be loaded into memory. File size determines storage infrastructure requirements.

2.4.2 Reproducibility Assessment

To assess measurement reproducibility, each compression implementation was benchmarked 10 times on each dataset. Results were aggregated to compute mean, standard deviation, minimum, maximum, median, and coefficient of variation (CV%) for all metrics. This multi-run approach quantifies both the deterministic nature of compression algorithms (compression ratio should be identical across runs) and the variability in timing measurements due to system-level factors such as CPU scheduling, cache state, and background processes.

All benchmarks were performed on a workstation with Intel Xeon processor, 64 GB RAM, and SSD storage running Linux. The absolute write/read times depend on system specifications; therefore, meaningful comparisons are relative performance differences between implementations on the same system rather than absolute timing values. The complete benchmark suite (12 HDF5 compression filters plus an uncompressed baseline $\times$ 3 chunking strategies $\times$ 5 datasets = 195 configurations, plus CSR, uint8 downcasting and direct numpy gzip on each dataset for 210 in all) was executed 10 times over a 24-hour period, yielding 2,100 individual measurements. The system was otherwise idle during benchmarking to ensure consistent performance measurements.

2.5 Implementation

Software Architecture—The benchmark was implemented in Python 3.11 using the following standard scientific computing libraries:

- NumPy (v2.3): Array operations and data manipulation
- h5py (v3.15): HDF5 file I/O
- SciPy (v1.16): Sparse matrix storage
- hdf5plugin (v6.0): Advanced compression filters
- matplotlib (v3.10): Visualization and plotting

Benchmark Workflow—The benchmark followed a systematic workflow for each compression implementation. First, the 4D dataset was loaded from EMD 1.0 format and analyzed for sparsity (zero fraction, value distribution, and statistics). For each implementation and chunking strategy combination, an HDF5 file was created with the specified compression and chunk size, the entire 4D dataset was compressed and written while recording write time and compressed file size. Read performance was then tested by loading the complete dataset to measure decompression speed. Compression ratios were calculated and results tabulated for further analysis. This entire process was repeated 10 times for each configuration to assess reproducibility.

Data Format—Input data uses the EMD 1.0 format, an HDF5-based standard with the structure `/version_1/data/datacubes/datacube_000/data`. This ensures compatibility with existing 4D-STEM analysis tools including py4DSTEM [20] and LiberTEM [21].

Reproducibility—Source code and detailed results are available at: <https://github.com/ondrejdyck/4d-stem-compression-benchmark-public>. The same repository holds the multislice simulation and the code-length calculations behind Figure 6 and Table 6; Supporting Information Section S3 gives the commands and the runtime.

Claude and ChatGPT were used for coding assistance and language editing; all scientific analysis, interpretation, and final manuscript content were reviewed by the authors.

3 Results

3.1 Overview of Compression Performance

We evaluated 13 off-the-shelf compression implementations across 5 datasets ranging from 8 MiB to 8 GiB, with sparsity levels from 49.5% to 92.8%. Each implementation was tested with three chunking strategies (real-space, balanced, and single-frame), yielding 39 HDF5-based configurations per dataset plus sparse and custom methods. Table 3 summarizes the key characteristics of each dataset and the best compression achieved.

Throughout the Results, we treat these lossless, drop-in implementations as a practical baseline for file-based workflows; broader implications for throughput-driven data reduction are discussed in the Discussion.

Compression ratios ranged from $4.8\times$ for the least sparse dataset (3D_EELS, 49.5% zeros) to $34.9\times$ for the most sparse dataset (4D_EELS, 92.8% zeros). The median compression ratio across all implementations and datasets was $6.2\times$, demonstrating that 4D STEM data is highly compressible regardless of the specific implementation chosen.

Table 3: Dataset characteristics and best compression performance achieved. Chunking strategy is specified in parentheses where it differs from balanced.

Dataset	Size (GiB)	Sparsity (%)	Best Ratio	Implementation	File Size (MiB)
4D_EELS	2.0	92.8	$34.9\times$	blosc_zstd	58.6
4D_Diff	8.0	74.7	$10.2\times$	blosc_zstd (real-space)	803.2
4D_Diff-2x2-binning	4.0	70.9	$11.3\times$	blosc_zlib (real-space)	363.2
4D_Diff-4x4-binning	1.0	60.9	$6.9\times$	blosc_zlib (real-space)	147.6
3D_EELS	0.008	49.5	$4.8\times$	blosc_zlib	1.7

Figure 1 shows the compression ratios, write throughput, and read throughput achieved by the top-performing implementations across all datasets. Blosc-based implementations consistently outperformed traditional HDF5 compression filters, with blosc_zstd and blosc_zlib achieving the highest compression ratios in all five datasets under balanced chunking, and in 12 of the 15 dataset and chunking combinations overall.

3.2 Implementation Family Comparison

Blosc-based compression implementations demonstrated superior performance compared to traditional HDF5 built-in filters across all datasets. Table 4 summarizes the mean performance of each implementation family using balanced chunking for fair comparison, while Figure 2 provides a multi-dimensional comparison of the top-performing implementations.

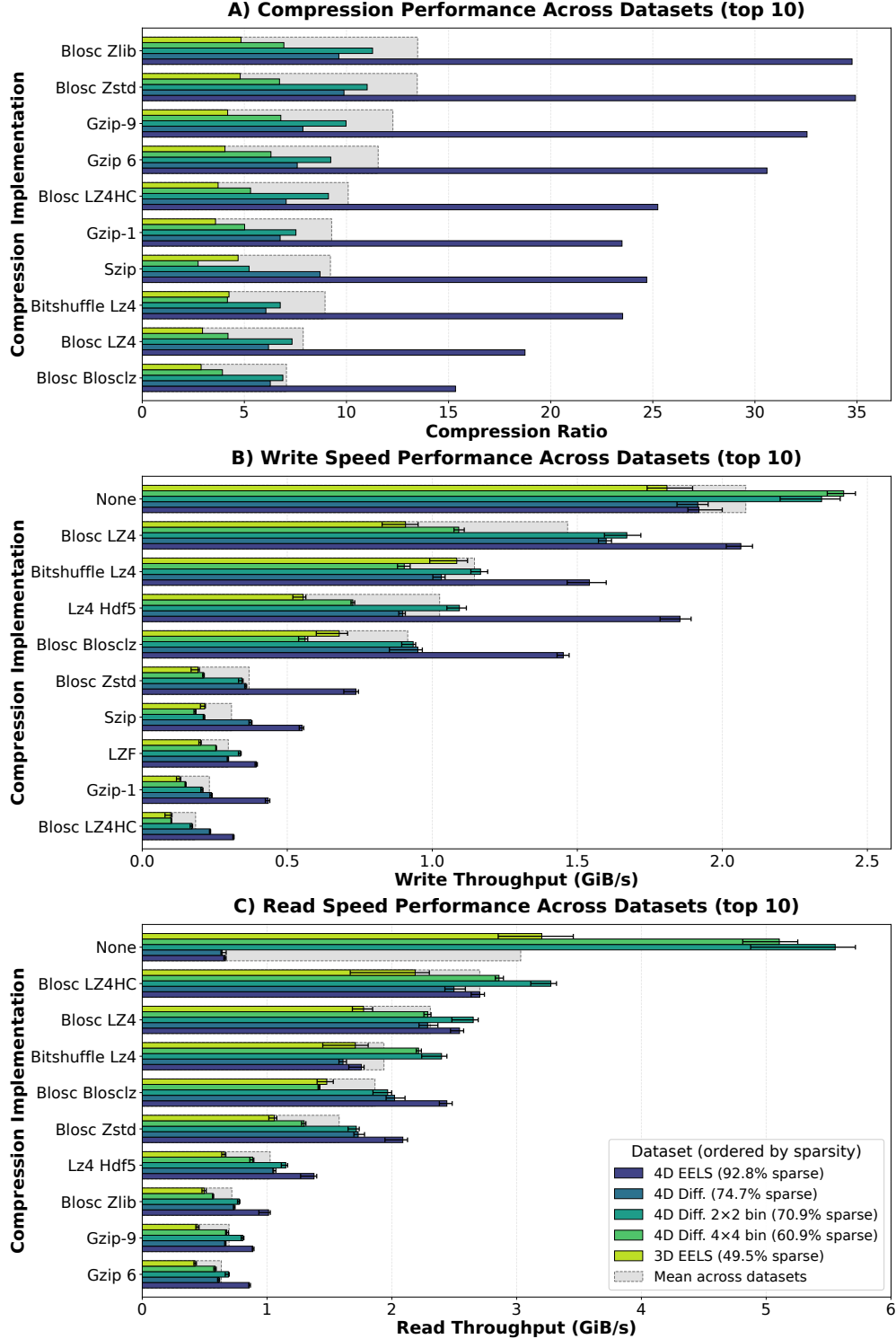


Figure 1: Cross-dataset performance comparison of the top 10 entries by each metric; the uncompressed baseline (None) appears in panels B and C for reference. (A) Compression ratios showing blosc_zstd and blosc_zlib achieving the highest compression across all datasets. Panel A has no error bars as compression ratio is deterministic (0% variation across 10 runs). (B) Write throughput demonstrating that Blosc-based implementations are significantly faster than traditional gzip compression. (C) Read throughput showing that Blosc implementations enable high-speed data access from compressed files. Error bars on panels B and C show minimum and maximum values across 10 benchmark runs. Bars show individual datasets ordered by sparsity; the gray bar shows the mean across datasets.

Table 4: Implementation family performance comparison (balanced chunking, averaged across all 5 datasets)

Implementation Family	Mean Ratio	Range	Mean Write (s)
Blosc (zlib)	13.5×	4.8–34.8×	19.3
Blosc (zstd)	13.5×	4.8–34.9×	8.3
Gzip-9	12.3×	4.2–32.6×	249.9
Gzip-6	11.6×	4.1–30.6×	47.4
Blosc (lz4hc)	10.1×	3.7–25.2×	14.8
Szip	9.2×	2.7–24.7×	9.8
Blosc (lz4)	7.9×	3.0–18.7×	1.9

Blosc-based implementations achieve compression ratios comparable to or better than gzip-9 while providing substantially higher write throughput, but it is useful to distinguish two different comparisons. First, for a like-for-like baseline focused on maintaining strong compression, blosc_zstd achieves compression comparable to gzip-9 (13.5× vs 12.3× mean for balanced chunking) while writing 19–69× faster across datasets and reading 1.9–2.6× faster. Second, for throughput-limited scenarios where sustained write performance is the dominant constraint, blosc_lz4 provides the fastest writing among the tested families, at the expense of lower compression. Relative to gzip-9, blosc_lz4 reduces write time by a factor of 87–324× across datasets (balanced chunking); summarizing the per-dataset speedups yields a typical improvement of $\sim 175\times$ (geometric mean). This throughput-first comparison is relevant for high-rate acquisition pipelines, where write performance directly impacts buffering requirements and the feasibility of near-real-time ingestion, even though software-only compression is unlikely to match the peak output rates of modern detectors.

The superior performance of Blosc implementations can be attributed to their use of byte-shuffling and blocking [14] together with a SIMD-accelerated shuffle [22], which are particularly effective for numerical scientific data. Because Blosc Zlib and gzip-9 apply the same underlying DEFLATE algorithm, the comparison between them isolates the effect of the surrounding implementation: Blosc Zlib achieves a 10% higher mean compression ratio (13.5× vs 12.3×) while writing 12.9× faster (19.3 s vs 249.9 s mean write time). The gzip implementations apply the algorithm directly to the byte stream, without the preprocessing that reorders numerical data to expose the redundancy the codec can then exploit.

Read performance also favored Blosc implementations. For the 8 GiB dataset, blosc_zstd achieved 1.7 GiB/s read throughput compared to 0.7 GiB/s for gzip-9, providing 2.6× faster decompression. This enables efficient data loading for analysis workflows where the entire dataset must be read into memory.

3.3 Chunking Strategy Analysis

Chunking strategy had minimal impact on compression ratio (typically <5% variation) and modest effects on read/write throughput. Table 5 summarizes the average performance by chunking strategy for the 8 GiB 4D Diff. dataset, averaged across all 13 compression implementations.

Table 5: Chunking strategy impact on performance (8 GiB 4D Diff. dataset, averaged across all implementations)

Strategy	Chunk Size	Mean Ratio	Mean Write (GiB/s)	Mean Read (GiB/s)
Real-space	(32, 32, 256, 256)	6.93×	0.73	1.67
Balanced	(16, 16, 128, 128)	6.67×	0.63	1.19
Single-frame	(1, 1, 256, 256)	6.88×	0.74	1.68

The compression ratio differences were negligible (6.67–6.93×, <4% variation). Write and read throughput showed modest variation (write: 0.63–0.74 GiB/s, $\sim 17\%$ range; read: 1.19–1.68 GiB/s, $\sim 40\%$ range), but these effects were implementation-specific with large variation within each chunking strategy (standard deviation 0.7–1.2 GiB/s for read). This indicates that chunking optimization provides limited benefit and would need to be tailored to specific compression implementations.

Figure 3 shows the trade-offs between chunking strategies across the six key implementations. Compression ratios vary modestly across chunking strategies. Read and write throughput show modest variation with

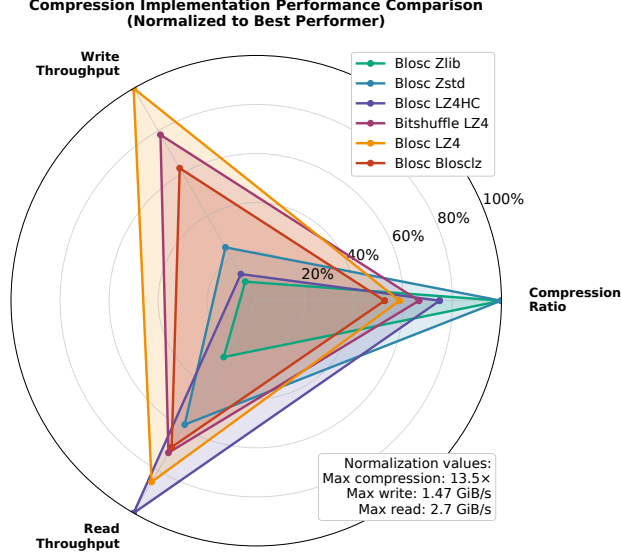


Figure 2: Multi-dimensional performance comparison of representative compression implementations. The radar chart shows normalized performance across three metrics: compression ratio, write throughput, and read throughput. Six key implementations are shown: five that appear in the top 10 for all three metrics across all datasets, plus `blosc_zlib`, which matches the highest compression ratio despite slower write speed. Blosc-based implementations (`Blosc Zstd`, `Blosc Zlib`, `Blosc LZ4`, `Blosc LZ4HC`, `Blosc BloscLz`) achieve well-balanced performance across all dimensions. The ideal implementation would reach the outer edge in all three dimensions. Based on aggregated statistics from 10 benchmark runs.

chunking strategy, with implementation-specific effects. The variation within each implementation across chunking strategies is comparable to or smaller than the variation between implementations, indicating that chunking effects are secondary to the choice of compression implementation itself.

3.4 Sparse Matrix and Custom Compression Methods

Sparse matrix storage (CSR format) and ad hoc data-reduction strategies did not outperform standard HDF5 compression implementations. Across the five datasets, CSR storage provided modest and inconsistent size reduction ($0.63\times$ to $4.60\times$), reflecting both index/metadata overhead and the lack of large contiguous zero blocks in typical diffraction patterns [18]. Likewise, the `uint8+overflow` strategy yielded limited and dataset-dependent compression ($1.53\times$ to $4.00\times$) and remained substantially worse than Blosc-based methods on the highly sparse datasets.

Two illustrative examples highlight this behavior. For the 2.0 GiB 4D_EELS dataset (92.8% zeros), CSR achieved $4.60\times$ compression (445 MiB), whereas `balanced_blosc_zstd` achieved $34.92\times$ (58.6 MiB), producing a $7.6\times$ smaller file. For the 8.0 GiB 4D_Diff dataset (74.7% zeros), CSR achieved $1.30\times$ compression (6.13 GiB), while `real_space_blosc_zstd` achieved $10.20\times$ (803 MiB), again yielding a $7.8\times$ smaller file.

Overall, these results indicate that practical storage and I/O benefits for 4D-STEM datasets are best achieved using established compression implementations.

3.5 Binning Effect on Compression

Binning the diffraction patterns reduces uncompressed file size and can reduce sparsity, which may reduce compressibility for aggressive binning. For the 4D Diff. dataset series:

- No binning (256×256): 8.0 GiB, 74.7% sparse, $10.2\times$ compression $\rightarrow$ 803 MiB (`real_space_blosc_zstd`)
- 2×2 binning (128×128): 4.0 GiB, 70.9% sparse, $11.3\times$ compression $\rightarrow$ 363 MiB (`real_space_blosc_zlib`)

Effect of Chunking Strategy on Compression Performance

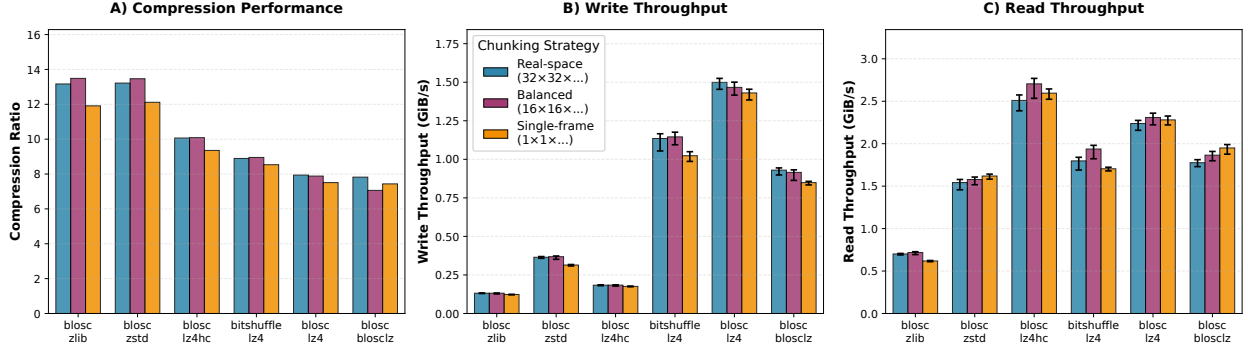


Figure 3: Effect of chunking strategy on compression performance for representative implementations. (A) Compression ratios show modest variation across chunking strategies. Panel A has no error bars as compression ratio is deterministic (0% variation across 10 runs). (B) Write throughput shows modest variation across strategies, with implementation-specific effects. (C) Read throughput shows modest variation across strategies, with implementation-specific effects. The variation within each implementation across chunking strategies is comparable to the variation between implementations, indicating that chunking effects are implementation-dependent. Error bars on panels B and C show minimum and maximum values across 10 benchmark runs. Six key implementations are shown (see Figure 2 caption for selection criteria). Three chunking strategies are compared: real-space (32×32 scan positions), balanced (16×16), and single-frame (1×1); detector-space chunk dimensions vary by dataset (Table 5). Results averaged across all five datasets.

- 4×4 binning (64×64): 1.0 GiB, 60.9% sparse, $6.9 \times$ compression $\rightarrow$ 148 MiB (real_space.blosc_zlib)

Binning reduces the final compressed file size, but it also discards information. In this dataset series, the 4×4 binned data compresses to 148 MiB, substantially smaller than the unbinned compressed data (803 MiB), but with reduced detector resolution. In our workflow, binning was performed by averaging and the resulting binned datasets were stored as `float32`, which changes bytes-per-pixel and the distribution of exact zeros relative to the original `uint16` data.

It is important to distinguish *compressibility* (compression ratio) from *final storage cost* (final file size). Binning reduces the uncompressed data volume, which generally decreases the final compressed file size even if the compression ratio does not improve. At the same time, averaging can reduce sparsity and alter the value distribution, which can reduce compressibility for aggressive binning. In our 4D Diff. dataset series, sparsity decreases from 74.7% to 60.9% between the unbinned and 4×4 binned data, and the best compression ratio decreases from $10.2 \times$ to $6.9 \times$. Nevertheless, combining binning with lossless compression yields the smallest absolute files. Here, we convert an 8.0 GiB dataset into a 148 MiB file through 4×4 binning and lossless compression (real_space.blosc_zlib), a total reduction of $\sim 55 \times$ in storage cost. This illustrates how lossy reduction (binning) and lossless compression can be combined to achieve order-of-magnitude storage savings when scientifically acceptable. For example, binning or cropping diffraction data may be adequate for producing conventional virtual bright-field (BF) or annular dark-field (ADF) images, but it may be undesirable for applications that demand maximum spectral or angular fidelity, such as high energy-resolution EELS or high-resolution ptychographic reconstruction.

3.6 Effect of Data Sparsity on Compression

Compression performance exhibited a strong non-linear relationship with data sparsity among the datasets stored as `uint16` (Figure 4).

The non-linear relationship can be understood from information-theoretic considerations. Let p_0 denote the probability that a pixel value is exactly zero (i.e., the sparsity, $p_0 = s$). The Shannon entropy [23] of the corresponding binary zero/non-zero indicator is the binary entropy function:

$$H_2(p_0) = -p_0 \log_2(p_0) - (1 - p_0) \log_2(1 - p_0) \quad (1)$$

For data stored as 16-bit integers (`uint16`), a sparsity-only upper bound on the achievable compression ratio is $16/H_2(p_0)$ (this ignores the additional entropy carried by the non-zero values and therefore represents an optimistic limit). At 50% sparsity, $H_2 \approx 1.0$ bit, yielding an upper bound of $16\times$. At 93% sparsity, $H_2 \approx 0.37$ bit, yielding an upper bound of $43\times$. Our empirical results approach this sparsity-only bound for highly sparse data. The gap widens for less sparse data, but this reflects a property of the bound rather than weaker compression performance. The bound only considers the zero/non-zero indicator and omits the entropy carried by the non-zero values themselves (i.e. the information content of the non-zero pixels). That omission is small for 4D_EELS, whose non-zero pixels are few and confined to an effective 12-bit range, and large for 3D_EELS, whose y-summed spectra are both denser and spread over the full `uint16` range.

Compression benefits increase dramatically at high sparsity. The least sparse dataset (49.5% zeros) compresses by only $4.8\times$, while the most sparse (92.8%) reaches $34.9\times$ —a 7-fold difference across a 43 percentage point change in sparsity.

Blosc-zstd maintains superior compression across all sparsity levels, while the gap between Blosc and gzip narrows at lower sparsity. This suggests that Blosc’s byte-shuffling and blocking optimizations are most effective for highly sparse data.

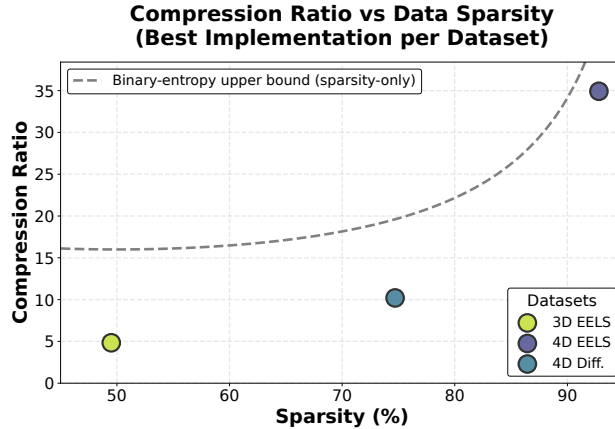


Figure 4: Compression ratio as a function of data sparsity for the three datasets stored as `uint16`. Each point represents the best compression achieved for a dataset (using Blosc Zstd or Blosc Zlib). The binned datasets are stored as `float32`, for which compression ratios are not directly comparable, and are omitted. The dashed line shows a sparsity-only binary-entropy upper bound ($16/H_2(s)$) based on the zero/non-zero indicator. The non-linear relationship demonstrates that compression benefits increase dramatically at high sparsity levels, with highly sparse datasets ($>90\%$ zeros) achieving compression ratios exceeding $30\times$.

The sparsity-only bound should not, however, be read as a measure of how close each implementation came to its limit. The bound understates the amount of information actually present, and it does so most severely for the least sparse data, where the non-zero values carry the most entropy. An accurate limit would have to account for the distribution of those values as well, and the obstacle there is fundamental rather than technical: an entropy bound of this kind requires the pixel values to be statistically independent, a condition satisfied only by data consisting of nothing but noise. Any structure—a diffraction disk, a spectral edge, an atomic lattice—appears precisely as correlation between pixels, and it is that correlation which leads us to posit an object in the data at all. A true lower bound on the compressed size of structured data can therefore be stated only once the objects it contains have been identified and represented, so that the true limit of lossless compression is not computable without interpretation. Simulated data offers one way past this obstacle, because there the objects are known from the outset. Section 4.3.3 carries out that calculation for a simulated 4D-STEM datacube. Blosc Zstd comes within a factor of 2.5 of that limit. The factor itself is specific to the dataset, but the implication is not: lossless compression is bounded by the entropy of the

source, so however much better an implementation becomes, the margin it can recover is a small multiple and not orders of magnitude.

There are two main takeaways here: (1) The compressor performance is already fairly well optimized. Improvements in lossless compression strategies will not yield the orders of magnitude that will be required to handle the data volumes expected from next-generation detectors. (2) The features of interest in the data are the correlations between pixels but noise is the real obstacle to efficient compression.

4 Discussion

4.1 Practical guidance for lossless compression

The results of this work provide immediate, practical guidance for managing 4D-STEM datasets with standard, widely available software. Across the five datasets considered here, Blosc-based implementations consistently occupy the favorable region of the compression–speed trade space, offering compression ratios comparable to or better than HDF5 built-in gzip while providing substantially higher write and read throughput. Within the Blosc family, different implementations provide distinct trade-offs. Blosc Zlib and Blosc Zstd achieve almost identical compression ratios, with Zstd markedly faster to write and read, and Blosc LZ4 offers the highest throughput at the cost of reduced compression. These trade-offs suggest that implementation choice should be guided by the dominant workflow constraint (storage footprint versus time-to-first-analysis versus sustained throughput).

Chunking strategy plays a secondary role compared to the choice of compression implementation. While chunk geometry can be tuned for specific access patterns and may yield modest gains for particular implementations, the aggregate effects on compression ratio and throughput are small relative to the differences among compression implementations. In most settings, a balanced chunking strategy provides a reasonable default. Further tuning may be advantageous when (i) a fixed access pattern dominates and (ii) a specific compression implementation is already selected.

4.2 Lossless compression is insufficient

Although lossless compression substantially reduces file sizes, it is unlikely to resolve the broader data-management challenge posed by rapidly increasing detector throughput. Contemporary pixelated detectors operate at frame rates approaching hundreds of kiloframes per second in electron microscopy workflows, and high-throughput pixel-array detectors in photon science routinely generate data streams on the order of tens of GBs^{−1}. The Timepix4 readout architecture, for example, supports multi-link high-speed output and motivates system designs targeting tens of Gbs^{−1} per chip [2, 3]. Likewise, kilohertz pixel-array X-ray detectors can stream at tens of GBs^{−1} and require hardware-accelerated acquisition pipelines to sustain such rates [4]. Commercial electron detectors similarly emphasize high-frame-rate operation and associated data handling constraints [5, 6]. In this context, improvements in lossless compression can reduce (but cannot eliminate) the mismatch between acquisition bandwidth and practical storage and I/O capacity.

4.3 Inference-sufficient representations

What is ultimately needed is not a better compressor but a criterion: a principled way to decide which of the data being acquired actually supports the inference an experiment is meant to support, and to discard the rest, possibly before it is even written. Researchers already make such decisions frequently, often intuitively, but in the coming era of autonomous and agentic science we will need to clearly articulate and likely encode into our algorithms the principles upon which we are relying to preserve relevant data.

At root, the data problem is not merely one of storing arrays efficiently. It is about making inferences. Detector outputs are best understood as measurements: sensor-mediated evidence from which we aim to infer properties of an underlying physical process. The data is serving a purpose and that purpose is a scientific inference. It is only in connection to such an inference that data obtains its relevance. We do not, for example, report every image collected during a microscopy session when we are reporting on a specific feature of a specimen. We select the few images (or spectra etc.) that are required to support the inference we intend to make. We do not store data from a ‘live view’. It is flashed on screen and discarded. This

evanescent data supplies us with background knowledge and helps guide our actions. At some point we hit ‘acquire’ and the data is stored. This is the data we may use to support an inference we intend to make.

It may be objected that the previous observations are so trivial that they are not worth stating. But there are two considerations that suggest they are worth dwelling upon. First, let us consider an atomically resolved image. To the human eye it is obvious where each atom is. We might object that pointing out where the atoms are is trivial. Hand the same image to a computer and one will discover what a challenge it is to say where the atoms are. It is not so trivial after all. We are at great pains to explain to a machine algorithmically what counts as an atom and what does not. As we develop increasingly autonomous systems, we will need to be able to articulate why we are licensed in making what seem to us to be trivial decisions. Is an agentic system licensed to discard ‘live view’ data? Why? The second consideration is that there is a persistent mantra among scientists that it is poor practice to discard data. Given that scientists routinely discard data (i.e. the ‘live view’ example) it is worth asking what the underlying motivation is for this mantra. There is something to it but it is too coarsely formulated. It is not that one should never discard data, but that one should not discard data that is relevant to the inference one intends to make.

We are faced with an increasing onslaught of data that cannot be stored. We are developing autonomous systems that will need to make decisions about what data to keep and what to discard. And what is critical about the data is its connection to scientific inferences.

4.3.1 What is an inference?

Because the term is central to what follows, it is worth being explicit about what is meant here by *inference*. We use it in the scientific sense rather than the computational one. We do not mean the evaluation of a trained model on new input, but the modes of reasoning by which observations are converted into claims about the world.

Following Peirce [24, 25], three modes of inference are conventionally distinguished, as outlined in Figure 5. The first is deduction, which takes a case and a rule and produces a result. We can think of this as the standard inputs and outputs of physical theory. We require objects and initial conditions (the case) and laws of physics (the rule) that make a forward prediction about what follows (the result). Deduction is the only inference of the three that is necessarily truth-preserving [26]. We might say that it is the only one that is truly an inference, in the sense that a true conclusion necessarily follows from true premises. The cost of this attractive truth-preserving feature is that it is non-ampliative. The result is specified within the rule and case. We can get the sense of this property by following Peirce’s example syllogism: (*g*) All the beans from this bag are white, (*f*) These beans are from this bag, therefore (*h*) These beans are white. We can see that the composition of the two relationships stated in *g* and *f*, if true, necessarily implies the relationship *h*. The relationship *h* has been stipulated by what is contained in *g* and *f*. As a visual aid, we may draw the relationships as a triangle, following the extended notes to mathematician David Corfield’s talk on this topic [27]: the nodes representing ‘these beans’, ‘bag beans’, and the property of being ‘white’; the relationships drawn as arrows (or mappings) between the nodes. This inference might then be written algebraically as $g \circ f = h$.

Induction takes a case and a result and attempts to infer a rule. It is the mode of reasoning by which we attempt to generalize universal laws from a finite set of observations. It is ampliative, in that it produces a rule that is not contained in the case and result. It is also tentative, in that the rule may be false even if the case and result are true. The syllogism might be written as (*f*) These beans are from this bag, (*h*) These beans are white, therefore (*g*) All the beans from this bag are white. The relationship *g* is not contained in *f* and *h*, but is an assumed (speculative) extension of the historically observed pattern. Critically, it is a testable extension. If we were to draw more beans from the bag and find a black one, the rule would be falsified.

Abduction takes a rule and a result and attempts to infer a case. It is the mode of reasoning by which we attempt to explain an observation by positing what is there. It is also ampliative, in that it produces content that is not stipulated by the rule and result. The case produced may not be unique and therefore represents a commitment to more than what was given. It is tentative. The case may be false even if the rule and result are true. The syllogism can be written as (*g*) All the beans from this bag are white, (*h*) These beans are white, therefore (*f*) These beans are from this bag. The relationship *f* is not contained in *g* and *h*, but is an assumed (speculative) explanation consistent with the observed pattern.

Inference	Given	Finds	The beans syllogism	As a triangle
Deduction <i>forced</i> <i>truth-preserving</i>	Rule + Case	Result	g All the beans from this bag are white. f These beans are from this bag. $\therefore h$ These beans are white.	
Induction <i>ampliative</i> <i>finds the rule</i>	Case + Result	Rule	f These beans are from this bag. h These beans are white. $\therefore g$ All the beans from this bag are white.	
Abduction <i>ampliative</i> <i>finds the case</i>	Rule + Result	Case	g All the beans from this bag are white. h These beans are white. $\therefore f$ These beans are from this bag.	

Figure 5: Peirce’s three modes of inference, illustrated with the classical example of beans drawn from a bag. Each mode is given two of the three terms—rule, case, and result—and finds the third. Deduction is truth-preserving but supplies none of its own premises; abduction and induction are ampliative and supply them. In a microscopy context the rule is the physics relating a specimen to a diffraction pattern, the case is the specimen posited to be present, and the result is the recorded data. Positing a case is only a posit; it becomes an explanation when deduction, run forward, reproduces the observation.

Both ampliative modes reach past the data, and they reach in different directions. An abduction posits a case. That case, with the rule held fixed, is a generator. Run deduction forward from it and it yields the observation together with indefinitely more that were not recorded, the same specimen at a finer sampling or seen from another angle. The content an abduction adds is that surplus, which is also why a record built from one is smaller than the record it replaces, as we will discuss later. A generator is shorter than its output. An induction reaches outward instead. It posits a rule, and the rule applies to cases never examined. The beans make the difference plain. “These beans are from this bag” is a claim about the beans in hand. “All the beans from this bag are white” is a claim about unobserved beans.

An instrument combines the two. Certifying a measuring device is an induction over a class of cases. The claim is that the device response is constant across time under the given conditions. No finite set of calibrations establishes it, and none could. Once it is committed to, however, each subsequent measurement affords the possibility of abductive interpretation.

Only deduction is truth-preserving, and it assumes the rule and the case are given; the two ampliative modes supply them, each requiring the other’s output as input and neither holding by necessity. The tentative, or speculative, nature of abduction and induction is what motivates a deductive check comparing our model predictions against observations. A measurement is scientifically valuable insofar as it supports this cycle, and a stored record is sufficient insofar as it can still support it.

4.3.2 What a measurement discards

Measurement is selective by construction. It assigns numbers to some property of the world and is silent about the rest, and what it retains is retained because it bears on a question someone wanted answered. A ruler reports length and says nothing about composition, temperature, or hardness. Measurement theory states the point formally: a measurement is a structure-preserving map (a homomorphism) from relations among objects to relations among numbers, and that map is many-to-one not one-to-one (an isomorphism)[28]. Distinct objects share a numerical description whenever they agree in the attribute measured. Thus, two different rods may share the same measured length. The measurement does not preserve every attribute

regarding the physical world from which it is drawn. Which attribute is recorded—and so what is discarded—follows from the inference the measurement is meant to support: length is sufficient to decide whether a part will fit, and nearly useless for deciding whether it will bear load.

Viewed this way, measurement already discards information in a concrete and familiar sequence of steps. Let’s consider a pixelated detector as our measuring device. First, integration over a finite exposure time collapses the full time history of the sensor response into a single value per pixel per frame. Second, digitization quantizes that analog value into discrete levels (e.g., a 16-bit integer), discarding variation smaller than one quantization step. Third, common rate-reduction operations such as windowing, binning, and bit-depth selection further reduce the number of reported samples while still producing the same kind of object (smaller frames or smaller datacubes). These operations do not attempt to identify “objects” in the data; they reduce volume by sampling the signal more coarsely.

At higher levels of reduction, instruments increasingly return interpreted quantities rather than minimally processed samples. A lock-in amplifier provides an instructive analogy: instead of storing the full waveform, it isolates a narrow frequency component assumed to contain the signal of interest and reports that component while discarding the rest. Compressive sensing provides a further example of inference-oriented reduction: rather than recording a fully sampled signal, acquisition is designed so that a reduced set of measurements is sufficient to recover a target representation under explicit structural assumptions (e.g., sparsity in a chosen basis) [29, 30, 31, 32]. From this perspective, the goal is not to compress a fully acquired dataset, but to avoid acquiring measurements that are not needed for the intended inference.

4.3.3 When a lossy reduction is sufficient

In the following discussion it will be helpful to have a concrete example of a specific kind of dataset. Here, we will use a simulated 4D-STEM (diffraction) dataset, shown in Figure 6. To acquire this kind of dataset, a focused electron probe is rastered across a specimen (Figure 6(a)), and at each probe position the resulting diffraction pattern is recorded on a pixelated detector. The result is a 4D datacube: two spatial dimensions for the probe position and two for the diffraction pattern.

A simulation gives us access to ground truth that a real experiment cannot provide. The multislice calculation returns, for each detector pixel, the fraction of the incident beam that reaches it. A beam current of 60 pA and a dwell of 1 μ s put a mean of 374.5 electrons into each probe position, and the two together give the expected number of electrons at each pixel, written λ (Figure 6(b)). What a detector records is a Poisson draw from λ (Figure 6(c)), simulating electron events. Each pixel receives an integer number of electrons, distributed by its own λ at that spatial position and independent of every other position, so a frame total varies from one repetition to the next; the frame shown collected 347. The specimen and the instrument determine λ , and λ together with Poisson statistics determines the counts. Everything the data can say about the specimen passes through λ .

A conventional annular dark field (ADF) image can be formed from the datacube by summing each diffraction pattern over a chosen angular range (Figure 6(d)). This reduces the 2D diffraction pattern at each probe position to a single number, producing a 2D image of the specimen.

Let us consider how much information was used to generate the datacube shown in Figure 6. A monolayer of WSe₂ is defined by its unit cell, atomic types, positions, and phonon amplitude; the instrument is defined by its accelerating voltage, convergence semi-angle, band limit, beam current, dwell time, real-space grid, slice count and scan grid. Storing this information requires just 347 bytes. Table 6 compares that against what the resulting cube costs to store.

The cube holds 128×128 probe positions on a 147×169 detector at a mean of 374.5 electrons per position and is 98.9% zeros (very favorable for compression). Blosc Zstd gives a compression of $84 \times$.

The model-based row asks a different question. What does the same cube cost to store if the compressor is handed the model that generated it?

Answering it takes one piece of vocabulary. A *code* is a rule for writing values as bits so that they can be read back, and any such rule must spend more bits on some values in order to spend fewer on others [34]. A rule that expects zeros can write a zero cheaply, but only by making a count of five expensive. The best rule is the one whose expectations match the data, and an arithmetic code realizes this almost exactly [35]: a value the rule assigns probability p costs about $-\log_2 p$ bits, so each halving of the probability costs one more bit. Those probabilities come from a model, which an arithmetic coder takes as a separate and replaceable

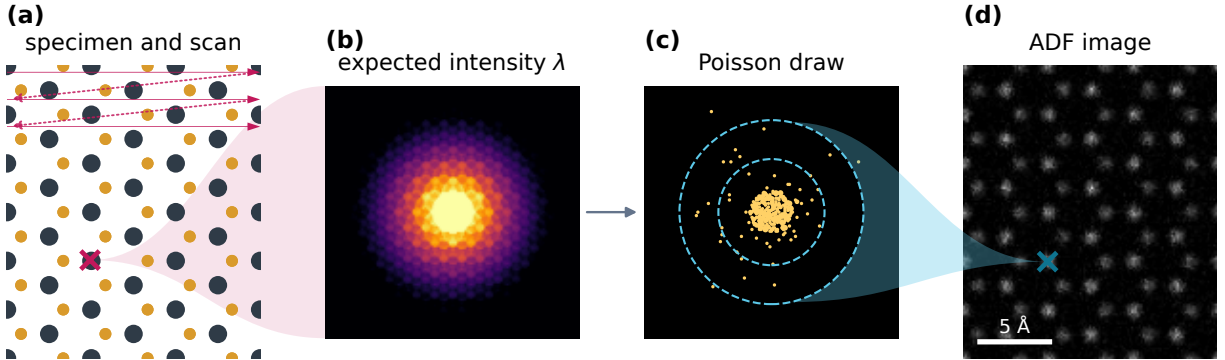


Figure 6: A simulated 4D-STEM dataset. (a) Diagram of the specimen, a monolayer WSe_2 with tungsten columns dark and selenium columns light; arrows indicate the raster and the cross marks an example probe position. (b) The expected intensity λ at that position, on a log scale. (c) A Poisson draw from (b), 347 electrons; dashed circles mark the annular detector, 75–130 mrad. (d) The ADF image formed by summing each frame over that annulus. Shaded connectors run from the marked probe position to its diffraction pattern, and from the annulus to the single number it becomes. Simulated with PySlice [33] at 80 kV with a 30 mrad convergence semi-angle, 60 pA and a 1 μs dwell. Full simulation parameters are given in Supporting Information Section S1.

Table 6: What it costs to store the simulated datacube of Figure 6, against what it cost to generate it. A representation that makes an assertion about the world can be false. The model-based reduction makes a claim about the specimen rather than the data; the other rows report data-derived quantities. Section 4.3.4 develops the distinction. How each entry was computed is given in Supporting Information Section S2.

Representation	Size	Reduction	Recovers	Asserts
Raw datacube (uint8)	388.2 MiB	—	—	—
Blosc Zstd, balanced chunks	4.6 MiB	84×	exactly	nothing
Model-based code, counts retained	1.8 MiB	210×	exactly	nothing
ADF image (float32)	64.0 KiB	6,211×	not at all	nothing
Model-based reduction, counts discarded	347 B	1,172,991×	not at all	the specimen

input. The general-purpose implementations above have a model too, but it is fixed and built in—that recently seen byte patterns recur—with no way to supply another.

Here the model is λ , which the 347 bytes are enough to reconstruct, and λ is the distribution each count was actually drawn from. For this cube a coder holding it needs 1.8 MiB to losslessly encode the data. That figure is not particular to the coder. The entropy of a Poisson distribution at each pixel’s own rate, summed over every pixel and probe position, is the shortest average length any lossless code can achieve on data from this source [23], and what was actually required here agrees with it to 2×10^{-4} . Nothing is discarded but nothing is asserted either. A model that is wrong—with a λ that is incorrect—simply produces a longer code. The data will decode exactly either way. What the 1.8 MiB reports is a code length rather than a file size, carrying none of the container and header overhead that any real compressor would require, and it is not an implementation anyone could deliver. Decoding means re-running the multislice, and λ is constructible here only because the specimen is known a priori. This gives us a bound on what a model of the source is worth while still insisting on lossless compression. Knowing the true generating model gives us another factor of 2.5. This is surprisingly small given that we have, in a sense, cheated by handing λ to the coder.

The ADF row reduces 24,843 detector pixels to one number per probe position, as the figure illustrates, achieving not just compression but true reduction in what data is kept. (The saving in bytes is smaller than the pixel reduction, since uint8 counts become a float32 value.) If the inference one is trying to support is the position and types of each atom within the field of view, then the ADF reduction of the 4D dataset may be sufficient. ADF detectors have been used for decades without anyone supposing that the paucity

of data supplied by them was insufficient to support the inferences that scientists sought to support. The inference-sufficiency of a representation is not a matter of how much data it contains, but whether it retains what is relevant to the inference at hand. The specific form of the reduction from a 4D-STEM datacube to an ADF image reflects the physics of electron scattering. It is a highly lossy reduction, ignoring the majority of the collected signal, but retains its inferential power because the reduction respects the physics.

Put another way, the rule (*g*) from Figure 5—the physics of electron scattering—is well-established. The data reduction from the full diffraction pattern to a single ADF value is a deterministic map that respects that rule, preserving our ability to make an abduction about the case—the sample—even in the face of a >6-thousand-fold lossy reduction in data volume.

The last row is the same 347 bytes with the coded counts thrown away, and an upper bound at that, since the parameters are serialized as minified JSON with the two random seeds excluded. We have a reduction of over a million because, in truth, no measured data has survived at all. What we have is the generating model. For these specific kinds of datasets, most of the time what we would like to extract from the data is this model and we do not care about the exact Poisson draw. The 347 bytes do not reproduce the cube. They reproduce λ , of which the cube is one draw among indefinitely many, and by the argument just given that is no loss in relevance, because the draw is the part that says nothing about the specimen. What the last row records is that every relevant fact about the (simulated) world held in a 388 MiB file can be written with a few hundred bytes.

This description is short only because it is written by someone who already has a model of the world that is known to be true. The experimentalist must make the measurement before the description can be written and without a priori knowledge about the world. Looked at in this light, we might say that interpretations of the world are attempts to account for sensory input with the shortest description: a generating model.

4.3.4 What a coarsening does not claim

A 4D datacube is itself a reduced record. The datacube discards the arrival time of each electron within the dwell. Event-based detectors attempt to preserve this time dimension as well, which we will discuss in more detail in Section 4.4. The ADF signal discards the angular distribution, reporting a single number per probe position.

It is worth being clear about what such a record does and does not say. An ADF value is the integrated intensity over its annulus, and it is that quantity exactly. It makes no claim about the bright field, which plainly does carry information—97% of the recorded electrons land within it, against 0.5% in the ADF annulus—and no claim about what happens on timescales shorter than the dwell. Those things are not denied; they are simply not being used.

A coarsening therefore cannot be false. A reader who takes an ADF image to assert that there are no lightly scattering atoms present—light atoms are increasingly difficult to detect in ADF images—has made a claim, and may be wrong, but the claim is theirs and the error is made in the interpretation. The record itself remains true no matter what is later concluded from it. In Table 6 this is represented by classifying the ADF record as asserting nothing.

What a coarsening risks instead is insufficiency: the quantity retained may not answer a question asked later. That is a failure of the judgment about what the record was for. The decision is thus a judgment about relevance to the intended inference. A monolithic ADF detector integrates over its angular range by construction. A pixelated detector preserves this scattering distribution, and an event list preserves the timing of each event. Whether those details are worth preserving is precisely the question at issue.

4.4 Interpretive representations

The model-based reduction with counts discarded—the last row in Table 6—appears to be an impossibility in practice. As we have already stressed, the only reason we can make such a reduction for our simulated 4D datacube is that we know a priori how the sample was configured. But let’s look at a concrete example that appears to be performing exactly this kind of reduction: event-driven detectors.

Event-driven detectors implement a form of *model-based reduction*. Rather than streaming a dense frame of outputs from every pixel, the detector (or upstream acquisition system) identifies an “event” and records only a small set of attributes such as time of arrival (ToA), time over threshold (ToT), and (x, y) coordinates.

In doing so, it discards the underlying sensor response that led to the event classification; the stored object is the interpretation (an event with attributes), not the full set of samples that could in principle have been recorded. And, importantly, it is not a coarsening of the data.

What makes such a step defensible is that no new physics is at stake in it. The rule—the physics by which a charge deposition produces a pulse of characteristic shape—is settled in advance, and is not what the experiment is investigating. The only unknown is the case: where and when an electron arrives. Confidence in the stability of that physics, and in the reliability of the abduction it licenses, is what permits the measurement trace to go unstored. For an event-driven detector this is not merely permitted but forced: as the next section shows, the trace could not be retained at any practical scale.

4.4.1 The evidence behind an event

What an event-based detector commits to is worth thinking through with some care. Figure 7 illustrates conceptually what happens along the way to the interpretation of an event. We have an array of pixels reading out data in parallel, Figure 7(a). Over the course of a single event we receive a single pulse in one of the pixels. This pulse, schematically shown in Figure 7(b), is assumed to rise above the noise and cross a configurable threshold. It recrosses the threshold on the way back to the dark current state. By recording the ToA, ToT, and the (x, y) pixel position we assert that we have captured the relevant properties of the received signal. We may then dispose of the entire analog trace, Figure 7(c), and retain only the interpretation of the event, Figure 7(d).

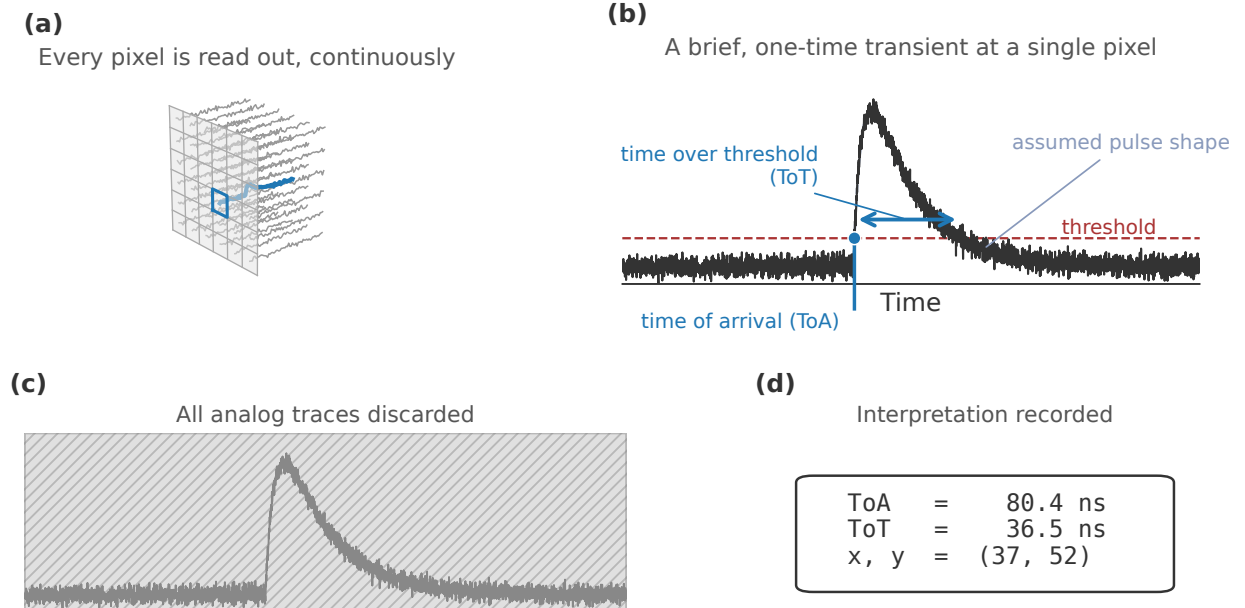


Figure 7: Event-driven detection as interpretive reduction. (a) Schematic of a pixelated detector reading out data in parallel. Each pixel is read out continuously, but the analog trace is not stored. During the time slice represented, one pixel, highlighted in blue, receives an electron event. (b) Schematic of the analog event trace. A transient crossing threshold is interpreted, against an assumed pulse shape, as the arrival of a single electron. (c) The analog trace is discarded. (d) What is recorded is four pieces of information (ToA, ToT, x, y) that are deemed to be relevant. This information is not a reduced sampling of the analog trace but an interpretation of it.

The first assertion we are making concerns the detector response. The $(\text{ToA}, \text{ToT}, x, y)$ record implies that a pulse of the expected shape crossing the threshold corresponds to a charge deposition of the reported magnitude, at the reported place and time. The second assertion concerns the environment. We presume that the objects depositing that charge are single electrons. Neither of these commitments is guaranteed. What justifies them is the prior hardware characterization and the environment within which it operates.

The detector sits inside an electron microscope and has been tuned to reliably detect electrons within some operational range. After calibration, we are confident enough in that control that we do not retain the data which would allow us to check the validity of each event.

The scale of what is discarded is worth making concrete. Suppose we have a 512×512 pixelated detector able to resolve arrival times to 200 ps, as the Timepix4 can [2]. What would it cost to preserve the dense measurement trace that licenses the conclusion that an electron arrived at pixel (37, 52) at time t ?

It would not be enough to store the trace of pixel (37, 52). The event record asserts not only that an electron arrived there, but that none arrived anywhere else at that instant. That is a claim about every other pixel, and the other pixels are not silent. Every pixel produces a continuous signal from which we must make an inference regarding the absence of an arrival. Such traces are precisely what one examines in order to decide where the threshold belongs in the first place. Evidence for the negative claim therefore requires the fully dense record across all 512×512 positions, at every time step.

Take a beam current of 60 pA. Electrons then arrive at about $3.7 \times 10^8 \text{ s}^{-1}$, one every 2.7 ns, or some 13 time steps apart at 200 ps resolution. Preserving the dense trace at a modest 8 bits per sample therefore costs approximately 3.5 MB for every electron detected, contrasted with the 64 bits in which such a detector records its interpretation of that electron. A 1 μs exposure, like what is simulated in Figure 6, yields about 374 events. This requires 1.3 GB of trace, or 3 kB of events. A 256×256 scan at that dwell would require some 86 TB of dense measurement while only 196 MB of event log, a factor of roughly 4×10^5 . Note that the cost of the dense trace does not depend on the beam current at all, whereas the event log shrinks as the dose is lowered.

The majority of that volume is the response of pixels where no electron arrived. Once the threshold has been set, that signal is classified as irrelevant and ceases to be recorded. It is not discarded because it is empty—it is not empty—but because it has been judged not to bear on the question being asked.

Event-driven detection is therefore not a compression strategy applied to a measurement. A compression strategy would seek to preserve the full measurement trace. Neither is it a lossy reduction through integration. A 4D-STEM datacube can be thought of as an integration through time; an ADF, an integration over scattering distribution. Event-driven detection produces an *interpretation*—an abduction—of the measurement trace on-the-fly and only this is preserved.

4.4.2 Delegating abductions

The step from trace to event asserts that the objects depositing charge are single electrons, that they produce a characteristic pulse shape, and that when such a pulse is encountered it can be interpreted as an electron count/event. This should be sharply distinguished from data reductions based on coarsening or integration. An interpretation can be false; a coarsening cannot.

There is an obvious risk in such a procedure. Recording a conclusion and discarding the evidence for it severs the deductive loop, illustrated in Figure 5. A model can still be run forward to predict what the instrument should have reported, but there is no longer an observation against which to compare that prediction; the result was discarded when the case was stored. This is the substance of the discomfort that attends discarding data in a scientific setting.

Why do we still feel confident in using an event-based detector? Within an electron microscope, all we expect to see are electrons. The abduction is licensed by a controlled environment with well-characterized equipment about which the only question we are asking is where and when the electrons arrived. In the vocabulary of Figure 5, the rule is given in advance by calibration and only the case is at issue. What makes this safe is that the population being interpreted is homogeneous with respect to the question being asked: one electron impact is like another, and that likeness was established independently of any specimen and has been validated prior to the automated abduction. Additionally, the abduction is not about the specimen, and the discarded data is not needed to support an inference about the specimen.

Confidence of this kind has to be paid for through prior characterization and certification. The arrangement is familiar: nobody retains the raw thermistor voltage from a calibrated thermometer, and nobody regards the temperature it reports as unscientific or poor practice because the raw voltage trace has not been preserved. The reading is trusted because the instrument was characterized in advance and is documented to have operated within the conditions under which that characterization holds. Discarding the evidence for an interpretation makes the same exchange, trading the ability to check each measurement for evidence that

checking was unnecessary.

That exchange is sound only when the certification is real. It requires that the ways the interpretation can fail be enumerated in advance, that each have either a signature detectable within the reduced representation or a validated operating envelope, and that the record carry evidence the instrument was within it. In the case of an event-driven detector there are additional interpretive complexities that we have omitted in our description. An electron event typically activates multiple pixels. This should not be interpreted as multiple electron events. Two electrons may impact the detector in rapid succession. This should not be interpreted as a single event. Cosmic radiation can impact the detector and these events should not be interpreted as electrons. Recognizing these possibilities and devising strategies for their discrimination and correction is the enumeration in advance that allows us to trust the interpretation.

4.4.3 Delegating inductions

Everything that licenses the event detector was settled before the experiment began. The harder question is whether such a license can ever be earned during an experiment, from what that experiment itself reveals. Suppose a one-micron region were surveyed on a grid of one hundred evenly spaced positions, and the same WSe_2 lattice was found at each position. One might then treat that lattice as the expected structure for the remainder of the session, so long as the stage stays within the surveyed region, and record only what departs from it. The certification would be established from the object under investigation, during the investigation.

Two quite different operations might be performed from this expected structure. The lattice could serve as a predictor: the expected pattern is subtracted and every residual retained, in which case nothing is discarded and nothing is asserted. That is simply compression with a better model of the source. One might think about lossless compression strategies that are developed on the fly, tailored to the specific content of the data being collected. Such a scheme has a property worth considering. Because nothing is thrown away, the length of the result measures the quality of the model that produced it. A model that fits badly assigns low probability to what actually occurred and pays for that in bytes, while the data survives intact regardless. Compressed size becomes a continuous check on the model, reported for every frame, requiring no ground truth and no separate validation.

The second, interpretive reading would assert the specimen structure and discard residuals below some threshold as noise. That threshold is a claim about what is not there, and it is the same claim the event detector makes, one level up.

The lattice model is a theory in the strict sense. It asserts that the region is WSe_2 with a given structure, and it forbids an observation, namely a frame that deviates beyond the threshold. Three things follow, and each depends on the inference being pursued.

First, the survey cannot establish the model. A hundred sites sampled from a one-micron region say nothing about what lies between. This is the problem of induction, and Popper's response was not to solve it but to deny that science needs it. A universal claim is never verifiable, because "we cannot search the whole world in order to make sure that nothing exists which the law forbids" [36, p. 70]. Our more modest, non-universal claim, is in principle verifiable across the specimen but we are specifically imagining a scenario where the data volume makes such an operation prohibitive. We can instead operate on falsificationist principles and search only for deviations from our model.

Second, what a mistake costs depends on what is being looked for. For a study of point defects the scheme might be close to ideal. The defect is precisely the residual, and it survives while the background is discarded. For a measurement of strain, however, a small deviation spread across many positions may fall entirely beneath the threshold and vanish; even though the data, the model and the threshold could be identical in both cases.

Third, the threshold fixes how much the model forbids. A region that is not WSe_2 at all would produce large residuals everywhere and announce itself, while a small departure would pass unrecorded. Raising the threshold narrows the class of potential falsifiers, which is another way of saying that it sets the size of error that can go unseen. Discard the residuals with nothing retained against them and that class is empty. The model is then beyond refutation, not because it is a poor theory but because the reduction destroyed the evidence that could have supported it.

Retaining a fraction of frames at full fidelity keeps the class populated. They must be chosen independently of the model rather than triggered by it, since a sample drawn from what the threshold already flags

returns what the model predicts it will return and confirms by construction. One percent of the frames would cost one percent of the volume. A thousand retained from a hundred thousand and found to fit proves nothing; a thousand that begin to fail is a reason to stop, examine the failures, and collect at higher density until the model is repaired or replaced. That is conjecture and refutation running inside a single experiment.

The aim here is not to prescribe any particular scheme but to help inspire new ways of thinking about the data reduction problem in light of the aims of science. We hope that the concrete examples given will help illustrate that the concept of interpretive reduction is already being leveraged in practice, that, in the case of the Timepix4, it affords a reduction of some five orders of magnitude, and that the principles behind it can be generalized to other contexts.

4.4.4 Representation as an experimental design variable

Storage capacity, bandwidth, and downstream I/O budgets are finite; allocating them to retain maximal detail limits the total volume of experiments that can be collected, retained, shared, and reprocessed. In high-throughput 4D-STEM regimes (and “big data” in general), the relevant decision is therefore not only *how* to compress data, but *which representation* to retain so that the stored record remains sufficient for the intended inference.

A practical implication is that representation should be treated as an experimental design variable. In some contexts, retaining fully dense, losslessly compressed data will remain appropriate. In other contexts, especially those that prioritize rapid feedback or sustained acquisition, it may be preferable to store reduced representations that preserve task-relevant information while discarding that which is deemed to be irrelevant.

To help make this idea operational, we summarize a simple framework for selecting and evaluating inference-sufficient representations. The intent is not to prescribe a single “correct” representation, but to begin thinking about what must be specified in order to justify an inference gaining support from a reduced representation. The first five apply to any reduction. The last two apply only where the record holds a conclusion rather than a coarsened measurement, since a coarsening cannot be false and an interpretation can. Which of the two applies depends on whether the interpretation reads the data in hand or extends a rule beyond it.

Design principles for inference-sufficient representations:

1. *Specify the inference target(s):* What decision, estimate, or hypothesis test must the stored record support?
2. *Specify the tolerable error:* What level of error would change the scientific conclusion?
3. *Identify the information that must be preserved:* Determine which invariants, features, or summary statistics are necessary to support the inference within the required uncertainty.
4. *Choose the least-burdensome representation that preserves them:* Fully dense measurement traces are one option, but not a requirement.
5. *Preserve provenance sufficient for trust and reproducibility:* Record calibration, processing assumptions, and metadata needed to reproduce the inference.
6. *Abduction—Certify an interpretation of the data in hand before discarding its evidence:* Interpreting a measurement is an abduction. Delegate abductions only where three things hold: what it asserts was settled in advance and is not what the experiment asks about; each way it can fail has a signature in the stored record or a validated operating envelope; and the record shows the envelope held.
7. *Induction—Keep refutable any rule extended beyond the data:* Extending a rule to what was not measured is an induction, and it can fail on data that the reduction discarded. Retain a fraction chosen independently of the model, and treat a failure among them as grounds for refutation and refinement.

4.5 Outlook

Taken together, these results support a two-level approach to data management. Lossless compression provides an important baseline for reducing storage and improving I/O without sacrificing the raw measurement

trace, and the present benchmark offers practical guidance for selecting among widely available implementations. However, the long-term sustainability of high-throughput measurement will increasingly depend on combining lossless compression with deliberate choices of acquisition parameters and data representations, including real-time or near-real-time interpretation pipelines that store task-relevant summaries rather than fully dense raw measurements. More broadly, this points toward a richer representation landscape—from event streams to task-specific summaries and learned descriptors—selected to preserve the grounds for an interpretation. In throughput-limited regimes, pipelines may store either the grounds for an interpretation or, when appropriate, the interpretation itself.

5 Conclusions

We benchmarked 13 off-the-shelf compression implementations for 4D-STEM datasets spanning 8 MiB to 8 GiB and a wide range of sparsity, evaluating compression ratio, write throughput, read throughput, and reproducibility across 10 runs. Compression ratios varied strongly with dataset characteristics, with the highest ratios achieved for the most sparse datasets, establishing a reproducible baseline.

Across all datasets, Blosc-based implementations provided the best overall balance of compression and performance. Blosc Zlib and Blosc Zstd achieved effectively identical compression ratios ($13.5\times$ mean for both), with Zstd substantially faster to write and read. For throughput-limited scenarios, Blosc LZ4 offered the fastest write performance and among the fastest read performance at the cost of reduced compression.

Finally, as detector data rates continue to increase, lossless compression alone will not eliminate storage and I/O constraints. Lossless compression preserves a measurement trace at a chosen point in the acquisition chain, but sustainable high-throughput acquisition increasingly requires selecting representations that are sufficient for the intended inference. In this view, compression is a valuable baseline. However, longer-term scalability will depend on combining efficient lossless compression with deliberate representation choices (including model-based reduction where appropriate) to maximize scientific value per stored byte.

Acknowledgments

This work was supported by the U.S. Department of Energy, Office of Science, Basic Energy Sciences, Materials Sciences and Engineering Division and by the Oak Ridge National Laboratory’s Center for Nanophase Materials Sciences (CNMS), a U.S. Department of Energy, Office of Science User Facility. AYB was sponsored by the Laboratory Directed Research and Development Program of Oak Ridge National Laboratory, managed by UT-Battelle, LLC, for the US Department of Energy.

References

- [1] Colin Ophus. Four-dimensional scanning transmission electron microscopy (4D-STEM): From scanning nanodiffraction to ptychography and beyond. *Microscopy and Microanalysis*, 25(3):563–582, 2019.
- [2] X. Llopart, J. Alozy, R. Ballabriga, M. Campbell, R. Casanova, V. Gromov, E.H.M. Heijne, T. Poikela, E. Santin, V. Sriskaran, L. Tlustos, and A. Vitkovskiy. Timepix4, a large area pixel detector readout chip which can be tiled on 4 sides providing sub-200 ps timestamp binning. *Journal of Instrumentation*, 17(01):C01044, January 2022. doi:10.1088/1748-0221/17/01/C01044.
- [3] Jonathan Correa, Alexandr Ignatenko, David Pennicard, Sabine Lange, Sergei Fridman, Sebastian Karl, Leon Lohse, Björn Senfftleben, Ilya Sergeev, Sven Velten, Deepak Prajapat, Lars Bocklage, Hubertus Bromberger, Andrey Samartsev, Aleksandr Chumakov, Rudolf Rüffer, Joachim von Zanthier, Ralf Röhlberger, and Heinz Graafsma. TEMPUS, a Timepix4-based system for the event-based detection of X-rays. *Journal of Synchrotron Radiation*, 31(Pt 5):1209–1216, July 2024. doi:10.1107/S1600577524005319.
- [4] F. Leonarski, M. Brückner, C. Lopez-Cuenca, A. Mozzanica, H.-C. Stadler, Z. Matěj, A. Castellane, B. Mesnet, J. A. Wojdyla, B. Schmitt, and M. Wang. JungfrauJoch: Hardware-accelerated

- data-acquisition system for kilohertz pixel-array X-ray detectors. *Journal of Synchrotron Radiation*, 30(1):227–234, January 2023. doi:10.1107/S1600577522010268.
- [5] DECTRIS ARINA - Hybrid-pixel detector for 4D STEM applications. URL: <https://www.dectris.com/en/detectors/electron-detectors/for-materials-science/arina/>.
 - [6] Direct Electron Detection | Microed | DQE Detector. URL: <https://www.dectris.com/en/detectors/electron-detectors/for-materials-science/quadro/>.
 - [7] Philipp M. Pelz, Ian Johnson, Colin Ophus, Peter Ercius, and Mary C. Scott. Real-Time Interactive 4D-STEM Phase-Contrast Imaging From Electron Event Representation Data: Less computation with the right representation. *IEEE Signal Processing Magazine*, 39(1):25–31, January 2022. doi:10.1109/MSP.2021.3120981.
 - [8] Electron Energy Loss Spectroscopy | MerlinEELS. URL: <https://quantumdetectors.com/products/merlineels/>.
 - [9] Matus Krajnak. Matkraj/read_mib, April 2021. URL: https://github.com/matkraj/read_mib.
 - [10] Benjamin H Savitzky, Steven E Zeltmann, Alexandra Bruefach, Alexander Rakowski, Mary Scott, Matthew L Henderson, and Colin Ophus. EMD 1.0 and ‘emdfile’: An HDF5 / Python Interface. *Microscopy and Microanalysis*, 29(Supplement_1):721–723, August 2023. doi:10.1093/micmic/ozad067.356.
 - [11] Peter Deutsch. DEFLATE compressed data format specification version 1.3. Technical Report RFC 1951, 1996. URL: <https://www.rfc-editor.org/info/rfc1951/>.
 - [12] Marc Lehmann’s ”LibLZF”. URL: <https://oldhome.schmorp.de/marc/liblzf.html>.
 - [13] HDF5: HDF5 Datasets. URL: https://support.hdfgroup.org/documentation/hdf5/latest/_h5_d__u_g.html#subsubsec_dataset_filters_szip.
 - [14] Francesc Altèd. Why Modern CPUs Are Starving and What Can Be Done about It. *Computing in Science & Engineering*, 12(2):68–71, March 2010. doi:10.1109/MCSE.2010.51.
 - [15] Lz4/lz4: Extremely Fast Compression algorithm. URL: <https://github.com/lz4/lz4>.
 - [16] Yann Collet and Murray Kucherawy. Zstandard Compression and the application/zstd Media Type. Request for Comments RFC 8478, Internet Engineering Task Force, October 2018. doi:10.17487/RFC8478.
 - [17] K. Masui, M. Amiri, L. Connor, M. Deng, M. Fandino, C. Höfer, M. Halpern, D. Hanna, A. D. Hincks, G. Hinshaw, J. M. Parra, L. B. Newburgh, J. R. Shaw, and K. Vanderlinde. A compression scheme for radio data in high performance computing. *Astronomy and Computing*, 12:181–190, September 2015. doi:10.1016/j.ascom.2015.07.002.
 - [18] Y. Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, April 2003.
 - [19] Mike Folk, Gerd Heber, Quincey Koziol, Elena Pourmal, and Dana Robinson. An overview of the HDF5 technology suite and its applications. In *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases*, AD ’11, pages 36–47, New York, NY, USA, March 2011. Association for Computing Machinery. doi:10.1145/1966895.1966900.
 - [20] Benjamin H. Savitzky, Steven E. Zeltmann, Lauren A. Hughes, Hamish G. Brown, Shiteng Zhao, Philipp M. Pelz, Thomas C. Pekin, Edward S. Barnard, Jennifer Donohue, and Luis Rangel DaCosta. py4DSTEM: A software package for four-dimensional scanning transmission electron microscopy data analysis. *Microscopy and Microanalysis*, 27(4):712–743, 2021.

- [21] Alexander Clausen, Dieter Weber, Karina Ruzaeva, Vadim Migunov, Anand Baburajan, Abijith Bahuleyan, Jan Caron, Rahul Chandra, Sayandip Halder, Magnus Nord, Knut Müller-Caspary, and Rafal E. Dunin-Borkowski. LiberTEM: Software platform for scalable multidimensional data processing in transmission electron microscopy. *Journal of Open Source Software*, 5(50):2006, June 2020. doi:10.21105/joss.02006.
- [22] The Blosc Developers. What Is Blosc?, July 2025. URL: <https://blosc.org/pages/blosc-in-depth/>.
- [23] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, July 1948. doi:10.1002/j.1538-7305.1948.tb01338.x.
- [24] Charles S. Peirce. Deduction, Induction, and Hypothesis. In Morris R. Cohen, editor, *Chance, Love, and Logic*. Routledge, 1923.
- [25] Francesco Bellucci and Ahti-Veikko Pietarinen. Charles Sanders Peirce: Logic. *Internet Encyclopedia of Philosophy*. URL: <https://iep.utm.edu/peir-log/>.
- [26] Wesley Salmon. *The Foundations of Scientific Inference*. University of Pittsburgh Pre, September 1967.
- [27] David Corfield. Charles Peirce, Inference, and Category Theory, November 2025. URL: <https://ncatlab.org/davidcorfield/show/Charles+Peirce%2C+Inference%2C+and+Category+Theory>.
- [28] D. H. Krantz, R. D. Luce, P. Suppes, and A. Tversky. *Foundations of Measurement - Volume 1: Additive and Polynomial Representations*. Academic Press, Incorporated, 1971. URL: <https://books.google.com/books?id=sNCNAQAACAAJ>.
- [29] E.J. Candes, J. Romberg, and T. Tao. Robust uncertainty principles: Exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Information Theory*, 52(2):489–509, February 2006. doi:10.1109/TIT.2005.862083.
- [30] David L. Donoho. Compressed sensing. *IEEE Transactions on information theory*, 52(4):1289–1306, 2006.
- [31] S. Foucart and H. Rauhut. *A Mathematical Introduction to Compressive Sensing*. Springer New York, 2013. URL: <https://books.google.com/books?id=zb28BAAQBAJ>.
- [32] A. Béché, B. Goris, B. Freitag, and J. Verbeeck. Development of a fast electromagnetic beam blanker for compressed sensing in scanning transmission electron microscopy. *Applied Physics Letters*, 108(9):093103, February 2016. doi:10.1063/1.4943086.
- [33] Harrison A. Walker, Thomas W. Pfeifer, Paul M. Zeiger, Jordan A. Hachtel, Sokrates T. Pantelides, and Eric R. Hoglund. PySlice: Routine Vibrational Electron Energy-Loss Spectroscopy Prediction with Universal Interatomic Potentials. *npj Computational Materials*, July 2026. doi:10.1038/s41524-026-02222-7.
- [34] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, August 1991.
- [35] Khalid Sayood. *Introduction to Data Compression*. Morgan Kaufmann, October 2017.
- [36] Karl Raimund Popper. *The Logic of Scientific Discovery*. Basic Books, 1959.